can it be used in software regression testing?

Yes. Jev can be very useful in **software regression testing**, especially for CI failure triage, flaky-test detection, test selection, and UI test decision-making. It should classify and recommend; deterministic CI rules and test code should make the final build decision. [1] [2] [3]

Best regression-testing use cases

Use case	Jev decision	Automated action
CI failure triage	Classify a failed test as `regression`, `flaky`, `environment`, `test_data`, or `unknown`	Hold merge for regressions, quarantine flaky tests, check infrastructure for environment failures [1] [2] [3]
Flaky-test detection	Decide whether inconsistent results are true flakiness or a real intermittent bug	Open a stability ticket, mark the test as quarantined, or escalate for investigation [4] [3]
Failure ownership	Identify the likely owning team, service, or code area	Assign the failed test to the correct developer or team [1] [2]
Test selection	Score which tests are most relevant to a code change	Run only high-risk or impacted tests first; defer low-risk tests [5] [1]
UI test automation	Decide which visible element matches the intended action, or whether a page state supports the expected result	Click the selected element or continue only above a confidence threshold [6] [7]
Error classification	Decide whether a failure is an app bug, network issue, dependency outage, or test-data problem	Retry infrastructure failures, fix fixtures, or file a product bug [7] [2]
Visual regression review	Decide whether a visual difference is intentional or a defect	Auto-accept approved changes, request human review for ambiguous diffs [6] [1]
Test-maintenance review	Score whether a changed expectation is legitimate or masks a regression	Require review before updating snapshots or expected results [8] [9]

Example: CI failure triage

A practical pattern is:

1. **Apply deterministic rules first.**
   For example, if a test failed and passed on retry in the same run, classify it as flaky; if logs contain DNS, timeout, or 5xx infrastructure markers and application code did not change, classify it as environmental. [2]
2. **Send only ambiguous failures to Jev.**
   Provide the test name, error message, recent pass/fail history, changed files, and relevant

log excerpts.

3. **Ask a bounded question.**
   "Classify this failure as `regression`, `flaky`, `environment`, `test_data`, or `unknown`."
4. **Apply a policy.**
   For example:
   - `regression` → block merge and notify the code owner
   - `flaky` → quarantine and create a stability ticket
   - `environment` → notify DevOps
   - `test_data` → notify the test maintainer
   - `unknown` → require human review

This exact pattern is used by Jev-based CI triage tools, which avoid auto-rerunning or masking failures. [3] [2]

Example: flaky versus regression

For a checkout test that failed intermittently, Jev could receive:

```
test: checkout_payment_test
recent_results: pass, fail, pass, fail
changed_files: checkout_service.py
error: Timeout waiting for payment confirmation
infrastructure_markers: none
```

Jev might return:

```
failure_type: flaky
confidence: 0.86
```

Your CI policy could then quarantine the test and open a stability issue—but it should not silently mark the build green. [1] [3]

Regression-testing Jev itself

If Jev is part of your product, it also needs its own regression tests. Use recorded fixtures and expected decisions to test:

- Question construction and response parsing
- Decision boundaries and confidence thresholds
- Fallback behavior for timeouts, rate limits, and invalid responses
- Changes in model behavior against a pinned evaluation set
- Human review before accepting a changed baseline [8] [9]

Best practice

Use Jev for **classification, scoring, selection, and triage**, not for replacing assertions or automatically rerunning failed tests. Keep hard assertions in your test suite, use Jev to prioritize and explain failures, and always retain an audit trail of its decision and confidence.[6] [3] [1]

⁂

1. https://katalon.com/resources-center/blog/jev-for-qa
2. https://github.com/criguex/jev-ci-triage
3. https://github.com/JevForge/jev-flaky-detective
4. https://x.com/justmalhar/status/2101104324306743644
5. https://jevwiki.ai/wiki/concepts/use-case-map.md
6. https://testomat.io/blog/deterministic-ai/
7. https://katalon.com/resources-center/blog/what-is-jev-guide-for-qa-testers?hs_amp=true
8. https://befailproof.ai/jev/evals/regression-testing/
9. https://paths.grasp.study/modules/bf739394-5e5c-4889-93d6-a6b2a169ed01/lessons/16cac48c-1f8c-4c3e-b43b-61cac75f0273
10. https://typesafe.ai/blog/introducing-system-one-models-and-jev
11. https://www.linkedin.com/posts/carlossg_tested-typesafe-ai-jev-to-see-what-the-buzz-activity-7506443577577734145-kuZi
12. https://jev-typesafe.org/
13. https://github.com/AbdelStark/awesome-typesafe-jev
14. https://github.com/kenhuangus/jev-usecases
15. https://docs.typesafe.ai/concepts/use-case-map
16. https://actionbox.cloud/blog/typesafe-ai-jev-review/